The Legitimacy Layer: A Runtime Governance Architecture for Human Oversight in AI-Assisted Critical Decision Systems

Hamza Aarab[0009-0006-0061-3888]
Independent Researcher
Geneva, Switzerland
m.hamzaaarab@gmail.com

Abstract - Artificial intelligence systems increasingly support operational decision-making in safety-critical environments, yet existing governance frameworks primarily regulate AI before deployment and after incidents, leaving the runtime execution interval structurally unaddressed. This interval - the legitimacy gap - is the period in which AI-generated decisions may proceed to operational effect without active oversight, contestability, or escalation control. This paper introduces the Legitimacy Layer, a runtime governance architecture that addresses the legitimacy gap by operating as model-agnostic middleware between the AI decision system and the operational environment. The architecture evaluates four governance signals (uncertainty, impact, reversibility, and contestation) and routes each decision through a legitimacy controller to one of four execution directives (execute, execute_async, escalate, or halt), regulated by a four-state autonomy model (Normal, Reduced, Human Oversight Required, Safe Halt). By specifying the architectural conditions under which human oversight, contestation, and escalation become structurally enforceable at runtime, the Legitimacy Layer positions human inclusion as an architectural property of the execution pipeline rather than an external assumption. The architecture is analytically validated through logical trace across four adversarial governance conditions and demonstrated through a reproducible prototype implementation publicly available for independent verification, proposing that runtime governance for AI-assisted critical decisions can be formally specified as model-agnostic middleware prior to empirical deployment validation.

Keywords - *Runtime AI governance, Human oversight, Contestability, Legitimacy gap, Governance middleware, Decision autonomy control*

I. INTRODUCTION

Artificial intelligence systems are increasingly embedded in operational environments where decisions carry direct real-world consequences. Across operational environments characterised by time pressure, consequence asymmetry, accountability requirements, and operator dependency (including energy grid management, transport control systems, financial operations, and healthcare decision support), AI outputs no longer function solely as advisory analytics but may directly influence actions affecting infrastructure stability, economic activity, and public welfare [1], [2]. As AI systems move closer to operational control, ensuring that their decision processes remain governable within institutional structures becomes a critical engineering challenge.

In high-consequence environments, technical accuracy alone is insufficient to guarantee institutional trust. Human operators bring contextual judgment, accountability, and the capacity for reasoned intervention that automated systems cannot replicate in situations involving competing values or unanticipated circumstances [3], [4], [5], [6]. Trust in AI-assisted decision systems depends not only on predictive accuracy but on the structural assurance that a human can intervene, contest, or override a decision when conditions warrant. This assurance cannot be assumed, it must be architecturally supported.

Existing AI governance frameworks are predominantly structured around two phases: pre-deployment validation, including risk assessment, model testing, and regulatory compliance; and post-incident accountability, including auditing and corrective review [7], [8]. While these mechanisms serve important functions, neither operates during decision execution itself. The runtime interval, in which AI-generated decisions transition from model output to operational effect remains structurally unaddressed.

This creates what this paper terms the legitimacy gap: the interval between AI decision generation and operational execution during which no governance mechanism evaluates whether the decision should proceed, be escalated, or be contested, producing governance failures that are invisible until their consequences have already materialised.

The contributions of this paper are as follows. Runtime governance problem definition: formalisation of the legitimacy gap as a structural absence in current AI governance frameworks. Legitimacy Layer architecture: specification of a model-agnostic runtime governance middleware comprising eight components, decision intake interface, governance signal evaluator, governance mode manager, legitimacy controller, contestability interface, governance policy store, execution router, and audit log module. Controller routing mechanism: design of a deterministic legitimacy controller that routes decisions across four execution directives according to governance signals and current autonomy state. Governance mode state machine: definition of a four-state autonomy control model: Normal Autonomy, Reduced Autonomy, Human Oversight Required, and Safe Halt, with formally specified entry conditions, transition logic, and exit conditions. Evaluation methodology: development of a scenario-based stress testing framework evaluated across four adversarial governance conditions with defined pass and fail criteria. Prototype implementation: a publicly available reference implementation demonstrating the executability of Algorithm 1 and the governance mode state machine, with a reproducible 1,000-decision simulation reproducing Table IV [31].

To our knowledge, no prior work integrates runtime governance signal evaluation, deterministic decision routing, graduated autonomy control, and pre-execution contestability as a unified model-agnostic middleware architecture positioned between an AI decision system and its operational environment. The Legitimacy Layer specifies this combination as a concrete, formally defined system contribution.

The remainder of this paper is organised as follows. Section II reviews related work. Section III defines key concepts and the

2026 1st International Conference on Human Centric Artificial Intelligence (ICHCAI) | 979-8-3315-5118-6/26/$31.00 ©2026 IEEE | DOI: 10.1109/ICHCAI70183.2026.11607581

979-8-3315-5118-6/26/$31.00 ©2026 IEEE

Authorized licensed use limited to: Universiti Malaysia Sabah. Downloaded on September 23,2026 at 15:21:26 UTC from IEEE Xplore. Restrictions apply.

problem structure. Section IV presents the architecture. Section V describes the evaluation. Section VI discusses limitations and future work. Section VII concludes. A note on scope. The Legitimacy Layer addresses human-centric AI at the architectural level: it specifies the runtime conditions under which human operators and affected stakeholders are structurally included in consequential AI decisions (through escalation, contestation, and graduated autonomy) rather than assumed to be present by default. The architecture does not model human cognition, decision quality under load, or interface usability; these are human-factors concerns requiring empirical human-in-the-loop validation, identified as declared follow-on work (Section VI).

II. RELATED WORK

A. Human Oversight in AI

Human oversight in AI-assisted decision systems has been widely studied across human-computer interaction, supervisory control, and human factors research. Human-in-the-loop models position human judgment as an active component within the decision pipeline, requiring human input before consequential actions are taken [3], [4]. Human-on-the-loop models adopt a monitoring posture, in which humans observe automated system behaviour and retain the capacity to intervene when anomalies are detected. Supervisory control frameworks formalise the conditions under which operators assume authority from automated systems, drawing on decades of research in aviation, process control, and military command systems [9], [6]. This body of work establishes that meaningful human oversight requires structured intervention pathways, clearly defined authority boundaries, and mechanisms for transferring decision control under specified conditions [10], [11]. However, these frameworks establish the desirability of oversight without specifying the architectural mechanisms through which it is triggered, routed, or enforced during execution.

B. Contestability in Algorithmic Systems

Contestability has emerged as a significant concern in the algorithmic accountability literature, reflecting the recognised need for individuals and institutions to challenge automated decisions that affect their interests. The right to explanation establishes that affected parties should be able to understand and dispute the basis of automated decisions [12], [13]. Procedural fairness frameworks extend this principle by requiring that appeals and review mechanisms be accessible, timely, and capable of producing meaningful redress. Contestability has also been examined as a design property, with proposals for interface mechanisms that allow users to query, dispute, or request review of algorithmic outputs [14], [15], [16]. Despite this progress, existing contestability mechanisms are predominantly oriented toward post-hoc or legal processes, they provide channels through which decisions can be challenged after they have taken effect, through administrative review, judicial appeal, or regulatory complaint. They do not provide operational interfaces through which contestation signals submitted by affected stakeholders can influence routing logic before execution completes. The timing problem is structurally unresolved: by the time contestation is possible under existing frameworks, the decision has already produced operational consequences.

C. Runtime Monitoring and Safety

Runtime monitoring has been an active area of research in autonomous systems, safety-critical software engineering, and machine learning operations. Anomaly detection methods identify deviations from expected system behaviour during operation, flagging inputs or outputs that fall outside learned or specified distributions [18]. Out-of-distribution detection research addresses the specific challenge of identifying when a deployed model encounters inputs that differ significantly from its training distribution, a condition associated with degraded predictive reliability. Uncertainty quantification methods provide probabilistic estimates of model confidence at inference time, enabling downstream systems to assess the reliability of individual predictions before acting on them [19], [20]. Collectively, these methods provide the signal detection capability that runtime governance requires; they identify conditions under which autonomous execution may be inappropriate. However, runtime monitoring literature addresses signal detection but not governance routing logic: what the system should do when a signal crosses a threshold remains outside the scope of monitoring frameworks.

D. AI Governance Frameworks

AI governance has developed as a multidisciplinary field addressing the responsible development, deployment, and oversight of AI systems across their operational lifecycle. Lifecycle governance models structure oversight across phases from design and development through deployment and decommissioning, establishing accountability responsibilities at each stage [21], [22]. Pre-deployment compliance mechanisms, including conformity assessments, algorithmic impact assessments, and certification processes, evaluate systems against technical and ethical requirements before they enter operational use. Risk-based governance approaches classify AI systems by potential harm severity and apply proportionate oversight requirements accordingly, concentrating regulatory attention on high-risk applications [7], [17], [23]. Post-incident accountability mechanisms provide structured processes for investigating failures, assigning responsibility, and implementing corrective measures following identified harms [8], [24]. While these frameworks collectively constitute a sophisticated governance infrastructure, they bracket the runtime execution interval without addressing it. Compliance operates before deployment, establishing the conditions under which a system is permitted to operate. Accountability operates after incidents, establishing what went wrong and who is responsible. The execution interval, as the period during which AI-generated decisions are actively transitioning to operational effect, lies between these phases and is left ungoverned by current lifecycle frameworks [25].

Authorized licensed use limited to: Universiti Malaysia Sabah. Downloaded on September 23,2026 at 15:21:26 UTC from IEEE Xplore. Restrictions apply.

Recent work has begun to propose model-agnostic runtime enforcement layers, control-plane governance architectures, and policy-driven orchestration mechanisms for agentic and enterprise AI systems. Gaurav et al. [26] introduce Governance-as-a-Service, a modular policy-driven enforcement layer governing agent outputs at runtime with allow, warn, block, and escalate outcomes. However, it does not specify the internal routing logic determining which outcome applies under which signal conditions, does not define a system-level autonomy state machine constraining controller behaviour across operational modes, and does not provide a pre-execution contestability mechanism. The Legitimacy Layer addresses each of these gaps directly. Goyal et al. [27] propose a unified AI orchestration platform with explicit control-plane and data-plane separation, incorporating policy engines, sandboxed execution environments, and governance lifecycle management across enterprise deployments. Recent preprints [28], [29], [30] address related runtime and agentic governance concerns in adjacent problem spaces.

III. PROBLEM DEFINITION

A. Critical Decision Environments

The Legitimacy Layer targets a class of operational environments characterised by four structural properties that collectively distinguish it from general-purpose AI application contexts. First, decisions must be made within operational windows that preclude extended deliberation, the temporal structure of the environment compresses the interval available for evaluation, routing, and intervention. Second, errors produce consequences that are difficult or impossible to reverse, the asymmetry between the cost of incorrect action and the cost of delayed action creates conditions in which governance failures propagate before they can be detected or corrected. Third, decisions must be traceable, explainable, and attributable to responsible parties, accountability requirements impose structural constraints on how decisions are recorded, communicated, and reviewed. Fourth, human operators are present but may be overloaded, unavailable, or operating under incomplete information and the assumption of continuous, attentive human oversight cannot be relied upon as a governance mechanism in its own right.

These four characteristics (time pressure, consequence asymmetry, accountability requirements, and operator dependency) are not incidental features of specific deployment domains. A runtime governance architecture for these environments must function under time constraints, must handle the possibility of operator unavailability, must produce auditable records, and must be capable of restricting autonomous execution without requiring continuous human attention. Deployment across specific domains will require domain-specific threshold calibration, illustratively, a clinical decision support deployment might configure uncertainty thresholds against ensemble disagreement metrics, impact classes against clinical harm severity taxonomies, and reversibility windows against the time horizon within which a proposed intervention can be safely withdrawn.

As a concrete illustration: a clinical decision-support system recommending discontinuation of anticoagulation in a post-stroke patient, with the upstream model reporting an uncertainty score of 0.65, the decision classed HIGH impact, non-reversible within a 6-hour governance window, and no active contestation signal, would be routed by Algorithm 1 through Rule 5 (HIGH impact with uncertainty above the 0.6 high threshold) to ESCALATE, transferring the decision to an attending clinician for review before execution.

B. The Runtime Legitimacy Gap

The central problem this paper addresses is formally defined as follows. The legitimacy gap is the interval between the generation of an AI decision and its execution in the operational environment during which no active governance mechanism evaluates the decision's legitimacy or risk.

The consequences of this absence are characterised by three distinct failure modes. The first is silent execution failure, in which a decision executes outside acceptable governance boundaries without triggering any oversight response. The system produces and applies a decision that would, under any reasonable governance evaluation, have required escalation or deferral but because no runtime evaluation occurs, the failure is invisible until its operational effects materialise. The second failure mode is uncontestable execution, in which a decision produces operational effects before affected parties have any opportunity to challenge it. Even where contestability mechanisms exist in principle, their post-hoc orientation means that by the time a challenge can be submitted, the decision has already committed its consequences. The third failure mode is escalation collapse, in which conditions arise during execution that genuinely require human judgment, elevated uncertainty, high-impact irreversible action, ambiguous accountability boundaries but no escalation pathway is available, operable, or sufficiently specified to route the decision to a human operator within the available window.

These failure modes are observable in current AI-assisted pipelines across healthcare, financial, and infrastructure contexts, where the execution interval proceeds without active governance evaluation, the structural gap the Legitimacy Layer is designed to close.

C. Design Requirements

The problem analysis in Sections III-A and III-B yields five design requirements that any runtime governance architecture for critical AI decision systems must satisfy.

R1 - Runtime governance signal evaluation. The architecture must evaluate governance-relevant signals during the execution interval, not only before deployment or after incidents. Signals including decision uncertainty, impact severity, reversibility, and stakeholder contestation must be assessed at the point of execution.

R2 - Deterministic routing logic. The architecture must route decisions through a deterministic governance controller that produces consistent, auditable routing decisions based on defined signal thresholds. Routing logic must be transparent, reproducible, and traceable in the audit record.

Authorized licensed use limited to: Universiti Malaysia Sabah. Downloaded on September 23,2026 at 15:21:26 UTC from IEEE Xplore. Restrictions apply.

R3 - Graduated autonomy control. The architecture must support graduated reduction of AI autonomy in response to elevated risk conditions, without requiring binary human-machine switching. A governance architecture capable only of full autonomy or full human control is operationally impractical in time-pressured environments.

R4 - Operational contestability. The architecture must provide a mechanism through which contestation signals submitted during the execution window can influence routing decisions before effects become irreversible.

R5 - Model-agnostic deployment. The architecture must operate without modifying the upstream AI decision system, enabling deployment as middleware compatible with existing operational systems.

The Legitimacy Layer is designed to satisfy all five requirements. Section IV presents its full architectural specification, including component definitions, controller routing logic, and governance mode transition conditions.

These requirements assume governance signals are trusted inputs supplied by the upstream system or operator-configured policy layer. Signal trustworthiness and threshold calibration are deployment-time concerns identified as priorities for future work.

IV. THE LEGITIMACY LAYER ARCHITECTURE

A. Architecture Overview

The Legitimacy Layer is a runtime governance architecture designed to regulate how AI-generated decisions are executed within operational systems. Rather than modifying the internal logic of the upstream AI model, the architecture operates as model-agnostic middleware positioned between the AI decision system and the operational execution environment, directly satisfying the model-agnostic deployment requirement established in Section III (R5).

As illustrated in Fig. 1, the Decision Intake Interface captures AI system outputs and converts them into structured decision records. Four runtime governance signals (uncertainty score, decision impact class, reversibility flag, and contestation signal) are then evaluated at the point of execution, satisfying R1.

Routing directives are issued to the Execution Router, which directs the decision toward autonomous execution, deferred asynchronous execution, escalation, or operational halt. The eight internal components implementing this pipeline are specified in Section IV-B.

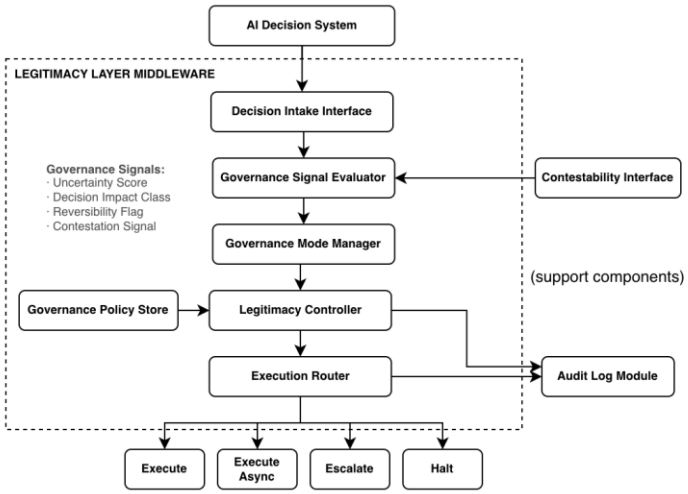


Fig. 1. Legitimacy Layer architecture showing decision flow from the AI decision system via the governance pipeline to execution or human oversight.

B. Component Architecture

The Legitimacy Layer is implemented as a set of coordinated runtime components that collectively regulate the execution of AI-generated decisions, transforming model outputs into governance-aware execution directives.

The Decision Intake Interface serves as the entry point to the Legitimacy Layer. It transforms raw AI model outputs into standardised decision records suitable for governance evaluation, enabling the architecture to operate independently of specific model implementations.

The Governance Signal Evaluator collects and validates the four governance signals associated with each decision record (uncertainty score, decision impact class, reversibility flag, and contestation signal) normalises them against policy-defined formats, and forwards the validated signal set to the Legitimacy Controller and Governance Mode Manager.

The Governance Mode Manager maintains the system's current autonomy state across four governance modes: Normal Autonomy, Reduced Autonomy, Human Oversight Required, and Safe Halt. Rather than treating autonomy as a binary condition, it dynamically constrains which routing directives the Legitimacy Controller may issue under prevailing operational conditions.

The Legitimacy Controller is the core routing mechanism of the architecture. It receives validated governance signals and the current governance mode, evaluates them against deterministic policy rules supplied by the Governance Policy Store, and issues one of four routing directives (execute, execute_async, escalate, or halt).

The Contestability Interface enables affected stakeholders to submit contestation signals challenging an AI-generated decision during the execution window before its effects become irreversible. Contestation signals are forwarded to the Governance Signal Evaluator and become active inputs to the controller's routing evaluation.

Authorized licensed use limited to: Universiti Malaysia Sabah. Downloaded on September 23,2026 at 15:21:26 UTC from IEEE Xplore. Restrictions apply.

The Governance Policy Store maintains policy thresholds and routing rules that parameterise the controller, enabling governance parameters to be updated without modifying controller logic. The Execution Router implements directives by directing decisions toward autonomous execution, reversible asynchronous execution, escalation, or halt with queue preservation. The Audit Log Module records all governance events in an append-only tamper-evident record operating independently of the controller. The deterministic routing logic is specified in Section IV-C.

C. Runtime Legitimacy Controller

The Legitimacy Controller is the deterministic routing mechanism at the core of the Legitimacy Layer. Its function is to evaluate governance conditions at the moment a decision transitions toward operational execution and determine which execution directive is permissible under the current runtime context. Routing is deterministic as identical signal inputs under the same governance mode always produce the same directive directly satisfying the deterministic routing logic requirement (R2) and enabling full post-hoc audit verification.

The controller evaluates four governance signals. The uncertainty score represents the predictive confidence of the upstream AI model and serves as an indicator of potential reliability degradation. The decision impact class reflects the severity of potential operational consequences, distinguishing routine actions from those capable of producing irreversible or system-critical effects. The reversibility flag indicates whether operational effects can be withdrawn within a defined execution window. The contestation signal represents an external challenge submitted by an affected stakeholder indicating that the decision's legitimacy requires re-evaluation before execution proceeds.

While the architecture treats these signals as validated inputs supplied by the upstream AI system or operator-configured policy layer, their practical production is domain-specific. In a clinical decision support context, for example, the uncertainty score may be derived from ensemble disagreement across model predictions or from conformal prediction intervals at inference time. The decision impact class may be configured by clinical governance teams against established harm severity taxonomies, distinguishing routine advisory outputs from recommendations capable of producing irreversible patient consequences. The reversibility flag reflects whether a proposed clinical action (such as a medication adjustment versus a surgical referral) can be withdrawn within a defined governance window. Specific configurations are governance policy inputs subject to domain-specific derivation rather than architectural constants.

Algorithm 1: route_decision(signals, governance_mode)

```
if governance_mode = SAFE_HALT then
    return HALT
end if
if governance_mode = HUMAN_OVERSIGHT_REQUIRED then
    return ESCALATE
end if
if signals.contestation_signal = TRUE then
    return ESCALATE
end if
if signals.impact_class = CRITICAL then
    return ESCALATE
end if
if signals.impact_class = HIGH and
    signals.uncertainty_score > high_threshold then
    return ESCALATE
end if
if signals.reversibility_flag = TRUE and signals.impact_class = LOW and
signals.uncertainty_score ≤ high_threshold then
    return EXECUTE_ASYNC
end if
if signals.impact_class = LOW and
    signals.uncertainty_score <  low_threshold then
    return EXECUTE
end if
return ESCALATE   ▷ Default: fail toward human oversight
```

The routing logic applies a strict priority ordering. Governance mode is evaluated first because it represents a system-level constraint that overrides all signal-level evaluation,a system in Safe Halt or Human Oversight Required mode cannot be returned to autonomous execution by a favourable signal combination on a single decision. Contestation signal is evaluated before impact class and uncertainty because it represents an external governance claim that cannot be overridden by system-generated signals alone. The default case escalates rather than executes, embedding a conservative failure posture: when no condition is matched, the architecture fails toward human oversight rather than autonomous action. Safety constraints override contestation signals, contestation overrides impact classification, and both override uncertainty-based routing conditions.

The rule set is deliberately non-exhaustive over the joint (impact × uncertainty × reversibility) signal space. HIGH-impact decisions with uncertainty at or below the high threshold, and LOW-impact decisions that fail either the reversibility precondition for EXECUTE_ASYNC or the low-uncertainty precondition for EXECUTE, are not matched by any preceding rule and therefore route through the default. This is consistent with the conservative failure posture: signal combinations not explicitly authorised for autonomous or asynchronous execution are routed to human oversight rather than resolved by a permissive default.

The asynchronous execution pathway introduced by the EXECUTE_ASYNC directive is described in Section IV-D.

D. Asynchronous Governance

Requiring human approval before every moderate-risk decision is operationally impractical. The EXECUTE_ASYNC directive addresses this by targeting decisions that cannot safely proceed under unrestricted autonomy yet do not require immediate escalation, providing a bounded execution pathway that maintains operational continuity while preserving governance

Authorized licensed use limited to: Universiti Malaysia Sabah. Downloaded on September 23,2026 at 15:21:26 UTC from IEEE Xplore. Restrictions apply.

intervention capability. This intermediate state implements the graduated autonomy control requirement (R3), positioning asynchronous governance between the execute and escalate directives established in Section IV-C.

When the Legitimacy Controller issues EXECUTE_ASYNC (triggered when the reversibility flag is TRUE, uncertainty remains within acceptable bounds, and no contestation signal is active) the decision executes immediately but enters a reversible execution window defined by governance policy. This window functions as a temporal buffer between execution initiation and final commitment of the decision's effects. The Contestability Interface remains active throughout: contestation signals submitted by affected stakeholders are processed by the controller, which responds by issuing ESCALATE and initiating reversal. If the window expires without contestation, the decision is committed and can no longer be reversed through the governance layer. All window events are recorded by the Audit Log Module.

Asynchronous governance is subject to strict constraints: it cannot be applied to decisions with a reversibility flag of FALSE, nor under governance modes that prohibit autonomous execution, Human Oversight Required or Safe Halt. The reversibility window is the primary operational interval in which contestability produces real governance effect. Once commitment occurs, runtime reversal is no longer possible through the architecture, making window integrity critical. The governance mode conditions governing all execution pathways are specified in Section IV-E.

E. Governance Mode State Machine

The Legitimacy Layer regulates system autonomy through a four-state governance mode model that constrains controller routing based on the broader operational condition of the system, not only on individual decision signals. This prevents favourable signal combinations on a single decision from re-enabling autonomous execution under elevated risk conditions. The Governance Mode Manager evaluates current mode as the first routing priority before any signal-level condition is assessed.

The architecture defines four governance modes. In Normal Autonomy, the system operates under standard conditions and the controller may issue execute or execute_async for decisions meeting signal thresholds, full routing capability is available. In Reduced Autonomy, the execute directive is restricted to low-impact decisions, asynchronous execution remains available for reversible decisions, and escalation is triggered when individual signal thresholds are met as the system continues operating under tighter governance constraints. In Human Oversight Required, both execute and execute_async are suppressed and all decisions are routed through escalate, transferring decision authority to a human operator. In Safe Halt, all execution is suspended, pending decisions are queued without processing, and only halt is permitted, activated by system integrity failures, audit log unavailability, governance policy store unavailability, or operator timeout on an unresolved escalation.

Mode transitions follow two directional paths, as illustrated in Fig. 2. The escalation path progresses from Normal Autonomy through Reduced Autonomy and Human Oversight Required toward Safe Halt as signal thresholds are crossed or integrity conditions are violated. Any mode may transition directly to Safe Halt on system integrity failure. Recovery transitions follow the reverse path but require explicit operator clearance at each stage, as the system does not return automatically to Normal Autonomy, ensuring that a human operator has confirmed that the conditions warranting reduced autonomy no longer apply. The evaluation of this state machine under adversarial conditions is presented in Section V.

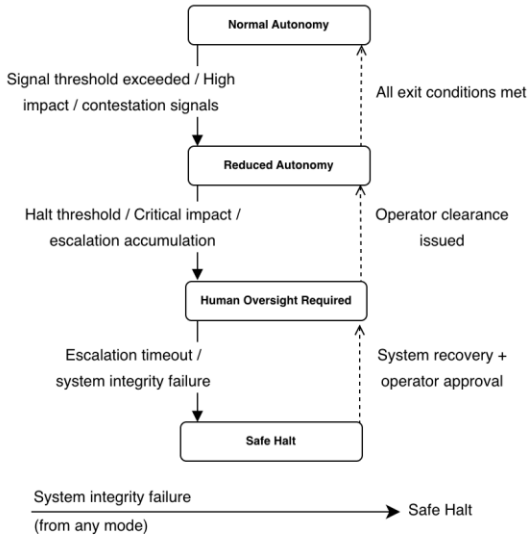


Fig. 2. Governance mode state machine showing escalation and recovery transition paths between the four autonomy states.

V. SPECIFICATION VALIDATION

A. Evaluation Approach

The architecture is evaluated using scenario-based analytical trace: governance logic is subjected to controlled adversarial runtime conditions and its outputs assessed against predefined pass and fail criteria. Scenarios are derived from the three runtime failure modes defined in Section III-B (silent execution failure, uncontestable execution, escalation collapse) in a manner analogous to hazard-based systems-safety analysis [32], with each scenario targeting one failure mode under adversarial signal injection; routing behaviour is then checked against predefined expected outcomes in the spirit of trace- and comparison-based validation in simulation modelling [33]. The approach is appropriate for pre-deployment specifications, evaluating correctness of governance signal interpretation, routing behaviour, and mode transitions without requiring a deployed implementation.

B. Evaluation Procedure

Each scenario initialises the system in Normal Autonomy mode, injects scenario-specific governance signal conditions, records the resulting controller directive and mode transition, and compares observed behaviour against predefined pass and fail conditions.

Authorized licensed use limited to: Universiti Malaysia Sabah. Downloaded on September 23,2026 at 15:21:26 UTC from IEEE Xplore. Restrictions apply.

C. Evaluation Scenarios

Each scenario is designed to stress-test the architecture's response to one of the three failure modes identified in Section III-B: Scenarios 1 and 2 address silent execution failure, Scenario 3 addresses uncontestable execution, and Scenario 4 addresses escalation collapse.

Scenario 1 - Uncertainty Spike. Injected signal: uncertainty score elevated progressively from below high-uncertainty threshold through to halt threshold across sequential decision records. Expected result: system transitions from Normal Autonomy to Reduced Autonomy; if halt threshold is reached, controller issues escalate; reversible decisions may route through execute_async under Reduced Autonomy. Pass condition: mode transition to Reduced Autonomy occurs at correct threshold; escalate issued before any halt-threshold decision routes to execute; all events recorded in audit log. Fail condition: controller issues execute for any decision at or above high-uncertainty threshold; mode transition does not fire correctly. Primary metric: escalation latency.

Scenario 2 - Sustained Uncertainty. Injected signal: uncertainty score held persistently above high-uncertainty threshold across a sustained sequence of decisions within a policy-defined observation window. Expected result: system remains in Reduced Autonomy; transitions to Human Oversight Required if escalation-triggering conditions accumulate; high-impact decisions must not route to autonomous execution. Pass condition: Reduced Autonomy entered and maintained correctly; Human Oversight Required triggered when accumulation conditions met; autonomous execution restricted for elevated-risk decisions. Fail condition: system returns prematurely to Normal Autonomy; high-impact decisions routed through execute during sustained abnormal uncertainty. Primary metric: missed escalation rate.

Scenario 3 - Stakeholder Contestation. Injected signal: contestation signal submitted against (i) a decision within an active reversibility window and (ii) a decision post-commit. Expected result: sub-scenario (i): controller issues escalate and initiates reversible execution intervention; sub-scenario (ii): signal logged, committed execution pathway unaltered. Pass condition: sub-scenario (i): escalate issued and intervention initiated before window closes; all events logged; sub-scenario (ii): post-commit contestation recorded, no intervention attempted on committed decision. Fail condition: sub-scenario (i): escalate not triggered within window; sub-scenario (ii): system attempts intervention on committed decision. Primary metric: contestation resolution time.

Scenario 4 - Operator Unavailable. Injected signal: escalation directive active; operator response withheld beyond policy-defined escalation timeout. Expected result: system transitions from Human Oversight Required to Safe Halt; pending decisions queued; no further execution directives permitted until recovery conditions satisfied. Pass condition: system enters Safe Halt at policy-defined timeout; all pending execution blocked; system re-enters Human Oversight Required (not Normal Autonomy) following operator re-authorisation; all events recorded in audit log. Fail condition: decision executes during Safe Halt; system returns directly to Normal Autonomy without passing through Human Oversight Required. Primary metric: operator workload.

TABLE I. Analytical Logical Trace Results Across Evaluation Scenarios

Scenario	Injected Condition	Observed Directive	Mode Transition	Pass/Fail
1 - Uncertainty Spike	uncertainty progressively exceeds high_threshold	ESCALATE at threshold crossing; EXECUTE_ASYNC for reversible LOW below threshold	Normal Autonomy → Reduced Autonomy	Pass
2 - Sustained Uncertainty	uncertainty held above high_threshold across window	ESCALATE for HIGH-impact; EXECUTE_ASYNC blocked above threshold	Reduced Autonomy → Human Oversight Required	Pass
3 - Contestation (in-window)	contestation_signal = TRUE, reversibility window active	ESCALATE issued; reversal initiated before commit	Human Oversight Required	Pass
3 - Contestation (post-commit)	contestation_signal = TRUE, window expired	Signal logged; no intervention on committed decision	No transition	Pass
4 - Operator Unavailable	escalation timeout exceeded	HALT issued; queue preserved; recovery requires Human Oversight Required not Normal Autonomy	Human Oversight Required → Safe Halt	Pass

D. Evaluation Metrics

Four metrics provide observable, architecture-specific indicators of governance correctness across the evaluation scenarios.

TABLE II. Evaluation Metrics - Observable Governance Correctness Indicators

Metric	Definition	Unit	Good Result	Failure Condition
Escalation latency	Time between detection of threshold-crossing condition and issuance of escalate directive	Seconds	**Within policy-defined bound**	Exceeds policy-defined bound
Missed escalation rate	Proportion of decisions meeting escalation conditions routed to execute or execute_async instead of escalate or halt	Percentage	**Zero missed escalations**	Any non-zero value
Contestation resolution time	Time between contestation receipt and final governance disposition	Seconds	**Resolved before window closes**	Not resolved before commitment
Operator workload	Number of escalation directives generated per decision window	Escalation per window	**Within policy-defined bounds**	Exceeds policy-defined bounds

The operator workload metric warrants additional consideration given its practical significance for deployment viability. As an illustrative estimate, if the system processes D decisions per hour and the simulation-derived escalation rate under balanced signal conditions is 82.9%, the expected operator load approaches $D \times 0.829$ escalations per hour under those conditions. In practice, this rate would be substantially lower under operationally representative parameter configurations, as characterised analytically in Section V-G and Table V; operational deployments would configure contestation frequency and impact class distributions to reflect domain-specific decision volumes and risk profiles. Research on human interaction with automation establishes that systems generating excessive escalation rates risk operator complacency and disuse: when automation interrupts too frequently, operators reduce their monitoring attention or disengage from escalation pathways entirely, undermining the governance function the architecture is designed to provide

Authorized licensed use limited to: Universiti Malaysia Sabah. Downloaded on September 23,2026 at 15:21:26 UTC from IEEE Xplore. Restrictions apply.

[6]. This workload estimate is illustrative and not derived from empirical deployment conditions; its validation requires human-in-the-loop testing identified as a priority item for future work.

E. Scenario Coverage Matrix

TABLE III. Scenario Coverage Matrix - Primary and Secondary Metric Applicability

Metric	Scenario 1	Scenario 2	Scenario 3	Scenario 4
Escalation latency	Primary	Secondary	Secondary	Secondary
Missed escalation rate	Secondary	Primary	-	Secondary
Contestation resolution time	-	-	Primary	-
Operator workload	Secondary	Secondary	Secondary	Primary

F. Evaluation Scope and Limits

This evaluation validates correctness of governance signal interpretation, routing logic, governance mode transitions, and architectural robustness under runtime governance disruptions. It cannot validate real-world deployment performance, statistical performance of the upstream AI model, operator usability under workload, organisational adoption, or threshold calibration for specific domains.

Four limitations constrain the conclusions: reliance on simulated signal injection rather than deployed execution; scenarios modelled as isolated rather than compound conditions; configurable rather than empirically derived thresholds; and operator workload treated as a design-constraint indicator rather than measured human-factors evidence. All four are revisited as future work in SectionVI-B.

G. Minimal Routing Simulation

To demonstrate the executability of the controller routing logic, a minimal simulation of 1,000 decision instances was conducted under Normal Autonomy mode assumptions. Governance signals were sampled across the full parameter space relevant to routing behaviour: uncertainty ∈ [0,1], impact ∈ {low, high, critical} with probabilities of 0.5, 0.3, and 0.2 respectively, reversibility ∈ {true, false}, and contestation ∈ {true, false} with a 0.2 probability of active contestation. Algorithm 1 was applied deterministically to each instance. Routing outcomes are summarised in Table IV.

TABLE IV. Routing Distribution Across 1,000 Simulated Decision Instce

Directive	Count	Percentage
Execute	53	5.3%
Execute_Async	118	11.8%
Escalate	829	82.9%
Halt	0	0.0%

Among high-impact and critical-impact instances, 529 of 529 (100%) were routed to escalate, confirming that the architecture maintains a conservative governance posture across varied signal conditions. The 82.9% escalation rate decomposes analytically across four routing sources: active contestation signals (20.0 points), CRITICAL impact decisions (16.0 points), HIGH impact combined with uncertainty above the high threshold (9.6 points), and the default fallthrough (36.4 points). The default fallthrough is the largest single contributor and reflects two classes of unmatched signal combinations: HIGH-impact decisions with uncertainty at or below the high threshold (14.4 points), which no preceding rule handles, and LOW-impact decisions that fail either the reversibility precondition for EXECUTE_ASYNC or the low-uncertainty precondition for EXECUTE (22.0 points). The rate is therefore not primarily driven by contestation but by the architecture's conservative failure posture specified in Section IV-C, which routes unmatched decisions toward human oversight rather than autonomous action.

To characterise how the escalation rate depends on input regime, Table V reports the analytical prediction of the total ESCALATE rate across a 2×2 grid of configurations, varying the uncertainty distribution and the contestation probability while holding the impact mix, reversibility probability, and controller thresholds constant. The values are derived in closed form from Algorithm 1's priority-ordered rules under the stated signal distributions. A verification run of the simulator at each grid cell (N=1000, averaged over 20 seeds) agrees with the analytical prediction within ±1.5 percentage points, confirming that the implementation faithfully realises the specified routing logic. The 82.9% result reported in Table IV corresponds to the top-left cell and agrees with its analytical value of 82.0%.

TABLE V. Total ESCALATE rate across input regimes (analytical prediction; simulation agrees within ±1.5 pp)

Uncertainty distribution	p_contestation = 0.20	p_contestation = 0.02
Uniform[0,1]	82.0%	78.0%
Beta(2,5)	69.2%	62.3%

(Impact mix LOW/HIGH/CRITICAL = 0.5/0.3/0.2, reversibility probability 0.5, thresholds low=0.3 / high=0.6.)

The grid shows that the escalation rate is a controllable function of input regime rather than a fixed architectural property: lower contestation probability reduces the contestation component by the corresponding amount, and a left-shifted uncertainty distribution reduces Rule 5 escalations while increasing HIGH-impact fallthroughs under the current rule set. These four configurations are illustrative of the parameter surface and do not identify a privileged operational rate; domain-specific rates depend on calibration of impact class distributions, uncertainty

Authorized licensed use limited to: Universiti Malaysia Sabah. Downloaded on September 23,2026 at 15:21:26 UTC from IEEE Xplore. Restrictions apply.

distributions, contestation frequencies, and thresholds against operational data, identified as a deployment-time concern in Section III-C and as future work in Section VI-B.

Halt directives were not produced under Normal Autonomy mode, consistent with the specification in Section IV-E, which restricts halt to Safe Halt mode transitions rather than signal-level routing alone. All routing decisions were consistent with the deterministic policy rules specified in Section IV-C, with no instance producing an unexpected directive. The prototype implementation and simulation code are publicly available to support reproducibility [31].
To illustrate dynamic governance mode behaviour, Table VI presents a manually constructed decision sequence designed to traverse all four governance states. Signal values are illustrative examples consistent with the architecture's transition logic; operational threshold values are governance policy inputs subject to domain-specific calibration, as noted in Section III-C.

TABLE VI. Deterministic Mode Transition Sequence

Step	Unc.	Impact	Rev.	Cont.	Directive	Mode Before	Mode After
1	0.3	Low	True	False	Execute	NA	NA
2	0.5	Low	True	False	Execute_Async	NA	NA
3	0.7	High	False	False	Escalate	NA	*RA*
4	0.8	High	False	False	Escalate	*RA*	*RA*
5	0.9	Critical	False	False	Escalate	*RA*	HOR
6	0.9	Critical	False	False	Escalate	HOR	HOR
7	-	-	-	-	Halt (timeout)	HOR	SH

*(*Unc. = Uncertainty, Rev. = Reversibility, Cont. = Contestation, *NA = Normal Autonomy, RA = Reduced Autonomy, HOR = Human Oversight Required, SH = Safe Halt.)*

The sequence confirms that the governance mode state machine transitions correctly under escalating risk conditions. This minimal simulation does not constitute empirical validation but demonstrates that the routing logic specified in Algorithm 1 is directly executable under concrete runtime conditions, and that the governance mode state machine behaves consistently with its formal specification.

VI. DISCUSSION AND LIMITATIONS

A. Discussion

An important boundary condition of this contribution should be noted. The architecture governs whether human operators are included in the decision process but does not address the quality of human judgment under the conditions in which escalation occurs. Research on automation-induced complacency [10], mode confusion [11], and operator performance under time pressure [5] shows that escalation does not guarantee a better outcome than autonomous execution in all conditions. Future work should examine the conditions under which human intervention improves governance outcomes and integrate human-factors evidence into escalation threshold calibration.

B. Limitations and Future Work

A reference prototype implementation has been developed and is publicly available [31]. The evaluation confirms architectural coherence under simulated signal injection but cannot capture failure modes that emerge only at runtime in real systems. Future work should evaluate the prototype's behaviour against real decision streams in a controlled operational context. The contestability interface assumes that affected stakeholders are identifiable, reachable, and capable of submitting contestation signals within the reversibility window. This assumption does not hold uniformly across deployment contexts — in emergency response or infrastructure control environments, affected parties may not be known at execution time or may be unable to act within compressed operational windows. Addressing this gap requires domain-specific stakeholder identification protocols and accessible interface designs, identified as prerequisites for deployment in contexts where real-time contestability cannot be structurally guaranteed.

The scenario-based evaluation validates routing correctness and mode transition behaviour but does not measure real-world latency, resilience under infrastructure load, or integration effects in complex deployments.

The routing and mode-transition rules depend on configurable parameters (uncertainty thresholds, escalation conditions, and reversibility windows)treated here as governance policy inputs rather than empirically validated constants. Future work should develop domain-specific threshold calibration methodologies and evaluate governance sensitivity to threshold variation.

Governance thresholds, impact class taxonomies, and contestation interface design are themselves value-laden decisions that redistribute risk between autonomous execution, operator workload, and affected stakeholder access; their domain-specific calibration therefore requires ethical review alongside technical calibration and is not a neutral engineering task.

The operator workload metric and the design requirements themselves require human-in-the-loop validation beyond the scope of this paper, including requirements elicitation from domain operators in the target deployment contexts. Real operational environments may also produce compound signal interactions (simultaneous uncertainty spikes, contestation events, and operator constraints) that exceed what the four isolated scenarios model, and should be addressed in future evaluation. The rule set of Algorithm 1 is specified to be non-exhaustive over the joint signal space, routing unmatched combinations to the default (Section IV-C); whether to replace this implicit default with explicit rules for currently-unmatched signal combinations is a design choice identified for future work.

Authorized licensed use limited to: Universiti Malaysia Sabah. Downloaded on September 23,2026 at 15:21:26 UTC from IEEE Xplore. Restrictions apply.

VII. CONCLUSION

Existing AI governance frameworks leave the runtime execution interval between decision generation and operational effect structurally unaddressed. To address this problem, the paper introduced the Legitimacy Layer, a runtime governance architecture designed as model-agnostic middleware positioned between an AI decision system and the operational environment. The architecture shows how governance can be embedded directly into the execution pipeline through governance signal evaluation, deterministic routing logic, graduated autonomy control, and operational contestability. By routing decisions through bounded directives (autonomous execution, reversible asynchronous execution, escalation, and halt)the Legitimacy Layer proposes that legitimacy can be treated as an operational control problem rather than only as a normative or regulatory concern. Future work should extend this contribution through operational deployment testing, domain-specific threshold calibration, and human-in-the-loop validation of escalation and oversight processes. Beyond these directions, the broader significance of this work lies in what it establishes for human-centric AI: that the moment a decision acts upon the world is not beyond governance, and that the systems entrusted with consequential decisions in critical environments can be held accountable not only before they are deployed or after they fail, but at the precise moment they act.

ACKNOWLEDGMENT

AI-assisted tools (including ChatGPT and Grammarly) were used for language editing and formatting. All scientific content, analysis, and conclusions are solely the author's responsibility.

REFERENCES

[1] J. Perez-Cerrolaza, J. Abella, F. J. Cazorla, C. Hernandez, R. Vargas, A. Batiz, P. Trigo, and A. Zuepke, "Artificial intelligence for safety-critical systems in industrial and transportation domains: A survey," ACM Comput. Surv., vol. 56, no. 7, Art. no. 176, 2024, doi: 10.1145/3626314.

[2] W. Wang and C. Chung, "Artificial intelligence in safety-critical systems: A systematic review," Ind. Manage. Data Syst., vol. 122, no. 2, pp. 442–470, 2021, doi: 10.1108/IMDS-07-2021-0419.

[3] T. A. Bach, J. K. Kristiansen, A. Babić, and A. Jacovi, "Unpacking human-AI interaction in safety-critical industries: A systematic literature review," IEEE Access, vol. 12, pp. 106385–106414, 2024, doi: 10.1109/ACCESS.2024.3437190.

[4] E. Veitch and O. A. Alsos, "A systematic review of human-AI interaction in autonomous ship systems," Safety Sci., vol. 152, Art. no. 105778, 2022, doi: 10.1016/j.ssci.2022.105778.

[5] T. B. Sheridan, Humans and Automation: System Design and Research Issues. Hoboken, NJ, USA: Wiley, 2002.

[6] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," IEEE Trans. Syst., Man, Cybern. A, Syst. Humans, vol. 30, no. 3, pp. 286–297, 2000, doi: 10.1109/3468.844354.

[7] B. W. Wirtz, J. C. Weyerer, and I. Kehl, "Governance of artificial intelligence: A risk and guideline-based integrative framework," Gov. Inf. Q., vol. 39, no. 4, Art. no. 101685, 2022, doi: 10.1016/j.giq.2022.101685.

[8] I. D. Raji et al., "Closing the AI accountability gap: Defining an end-to-end framework for internal algorithmic auditing," in Proc. 2020 Conf. Fairness, Accountability, Transparency (FAT*), Barcelona, Spain, 2020, pp. 33–44, doi: 10.1145/3351095.3372873.

[9] T. B. Sheridan, Telerobotics, Automation, and Human Supervisory Control. Cambridge, MA, USA: MIT Press, 1992.

[10] L. Bainbridge, "Ironies of automation," Automatica, vol. 19, no. 6, pp. 775–779, 1983, doi: 10.1016/0005-1098(83)90046-8.

[11] M. R. Endsley, "From here to autonomy: Lessons learned from human-automation research," Hum. Factors, vol. 59, no. 1, pp. 5–27, 2017, doi: 10.1177/0018720816681350.

[12] S. Wachter, B. Mittelstadt, and L. Floridi, "Why a right to explanation of automated decision-making does not exist in the General Data Protection Regulation," Int. Data Privacy Law, vol. 7, no. 2, pp. 76–99, 2017, doi: 10.1093/idpl/ipx005.

[13] B. Goodman and S. Flaxman, "European Union regulations on algorithmic decision-making and a right to explanation," AI Mag., vol. 38, no. 3, pp. 50–57, 2017, doi: 10.1609/aimag.v38i3.2741.

[14] T. Hirsch, K. Merced, S. Narayanan, Z. E. Imel, and D. C. Atkins, "Designing contestability: Interaction design, machine learning, and mental health," in Proc. 2017 Conf. Designing Interactive Syst. (DIS), Edinburgh, U.K., 2017, pp. 95–99, doi: 10.1145/3064663.3064703.

[15] D. K. Mulligan, D. N. Kluttz, and N. Kohli, "Shaping our tools: Contestability as a means to promote responsible algorithmic decision making in the professions," SSRN Electron. J., 2019, doi: 10.2139/ssrn.3311894.

[16] M. Almada, "Human intervention in automated decision-making: Toward the construction of contestable systems," in Proc. 17th Int. Conf. Artif. Intell. Law (ICAIL), Montreal, QC, Canada, 2019, pp. 1–10, doi: 10.1145/3322640.3326699.

[17] D. Amodei et al., "Concrete problems in AI safety," arXiv:1606.06565 [cs.AI], Jun. 2016.

[18] D. Hendrycks and K. Gimpel, "A baseline for detecting misclassified and out-of-distribution examples in neural networks," arXiv:1610.02136, 2016, doi: 10.48550/arXiv.1610.02136.

[19] Y. Gal and Z. Ghahramani, "Dropout as a Bayesian approximation: Representing model uncertainty in deep learning," in Proc. 33rd Int. Conf. Mach. Learn. (ICML), vol. 48, 2016, pp. 1050–1059.

[20] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 30, 2017, pp. 6402–6413.

[21] A. Dafoe, "AI governance: A research agenda," Future of Humanity Inst., Univ. of Oxford, Oxford, U.K., Tech. Rep., 2018.

[22] A. Batool, D. Zowghi, and M. Bano, "AI governance: A systematic literature review," AI Ethics, vol. 5, pp. 3265–3279, 2025, doi: 10.1007/s43681-024-00653-w.

[23] C. Novelli, F. Casolari, A. Rotolo, M. Taddeo, and L. Floridi, "AI risk assessment: A scenario-based, proportional methodology for the AI Act," Digit. Soc., 2024, doi: 10.1007/s44206-024-00095-1.

[24] C. Novelli, M. Taddeo, and L. Floridi, "Accountability in artificial intelligence: What it is and how it works," AI Soc., vol. 39, no. 4, pp. 1871–1882, 2024, doi: 10.1007/s00146-023-01635-y.

[25] J. Schuett, "Frontier AI developers need an internal audit function," Risk Anal., vol. 44, no. 11, Oct. 2024, doi: 10.1111/risa.17665.

[26] S. Gaurav, J. Heikkonen, and J. Chaudhary, "Governance-as-a-Service: A Multi-Agent Framework for AI System Compliance and Policy Enforcement," arXiv:2508.18765 [cs.LG], Aug. 2025, doi: 10.48550/arXiv.2508.18765.

[27] I. Goyal, G. Patil, and A. Shaikh, "Unified Multi-Channel AI Orchestration Platform Architecture," *J. Comput. Sci. Technol. Stud.*, vol. 7, no. 10, pp. 400–425, Oct. 2025, doi: 10.32996/jcsts.2025.7.10.42.

[28] C. L. Wang et al., "MI9: An integrated runtime governance framework for agentic AI," arXiv:2508.03858 [cs.AI], Aug. 2025.

[29] O. A. Basir, "The social responsibility stack: A control-theoretic architecture for governing socio-technical AI," arXiv:2512.16873 [cs.AI], Dec. 2025.

[30] E. Calboreanu, "LATTICE: A governance-first architecture for authorized autonomous AI operations," Zenodo, Tech. Rep., 2026, doi: 10.5281/zenodo.18419923.

[31] H. Aarab, "Legitimacy Layer Prototype," GitHub, 2026, v1.1. [Online]. Available: https://github.com/MrHamzaAb/legitimacy-layer-prototype

[32] N. G. Leveson, *Engineering a Safer World: Systems Thinking Applied to Safety.* Cambridge, MA, USA: MIT Press, 2011.

[33] R. G. Sargent, "Verification and validation of simulation models," *J. Simul.*, vol. 7, no. 1, pp. 12–24, 2013, doi: 10.1057/jos.2012.20.

Authorized licensed use limited to: Universiti Malaysia Sabah. Downloaded on September 23,2026 at 15:21:26 UTC from IEEE Xplore. Restrictions apply.